



Tecnicatura Universitaria en Programación a Distancia

Programación III

Trabajo Integrador Final: "FoodStore"

Alumno: Romero Abel Tomás - abeltomasr98@gmail.com | Comisión 5

Fecha de entrega: 02/07/2026

Enlace del repositorio: <https://github.com/AtomuDev/Programacion-3-Final>

Enlace del video: <https://youtu.be/hc0QptnujAw>

Índice

• 1. Marco teórico	3
• 2.1 Decisiones técnicas y arq.: Backend	3
• 2.2 Decisiones técnicas y arq.: Frontend	4
• 2.3 Decisiones técnicas y arq.: Estructura del proyecto	4
• 3.1 Captura de pantalla: Frontend	6
• 3.2 Captura de pantalla: Backend	13
• 4. Dificultades y soluciones	20
• 5. Herramientas utilizadas	20
• 6. Bibliografía / Webgrafía	20

1. Marco teórico

El proyecto **FoodStore** es una aplicación de gestión de ventas de productos, desarrollada como un sistema de dos capas independientes: un backend de consola en Java y un frontend web en TypeScript.

En el **backend** se utilizó Java 17 junto con JPA (Jakarta Persistence API) y Hibernate 6 como proveedor de persistencia, sobre una base de datos H2 embebida en archivo. Gradle se empleó como herramienta de build y gestión de dependencias, y Lombok para reducir el código repetitivo de las entidades (getters, setters, builders). JPA permitió mapear las clases del dominio (Categoria, Producto, Usuario, Pedido, DetallePedido) directamente a tablas relacionales, gestionando las relaciones entre entidades y el ciclo de vida de los objetos persistentes mediante el “EntityManager”.

En el **frontend** se utilizó Vite como bundler y servidor de desarrollo, junto con TypeScript para tipar el dominio de la aplicación (productos, pedidos, usuarios, roles) y reducir errores en tiempo de compilación. La interfaz se construyó con HTML y CSS organizados por página, sin frameworks de UI adicionales, priorizando una estructura simple y modular.

Como conceptos clave del desarrollo se aplicaron: persistencia orientada a objetos (ORM), separación de responsabilidades entre capas, reutilización de código mediante herencia y genéricos, y manejo de estado en el cliente mediante “localStorage”.

2. Decisiones técnicas y arquitectura

2.1 Backend

Se definió una clase abstracta “Base”, mapeada como “@MappedSuperclass”, de la cual heredan todas las entidades del dominio. Esta clase centraliza los campos comunes (id autogenerado, fecha de creación y el flag de baja lógica), para evitar duplicar esa lógica en cada entidad y garantizando que toda la jerarquía se comporte de forma consistente frente a altas, bajas y auditoría básica.

Sobre esa base se construyó un “BaseRepository<T extends Base>” genérico, que concentra las operaciones CRUD comunes a todas las entidades (guardar, buscar, listar, baja lógica). Cada repositorio específico (ProductoRepository, PedidoRepository, UsuarioRepository, CategoriaRepository) extiende esta clase y solo agrega las consultas propias de su dominio. Esta decisión evitó repetir la gestión del “EntityManager” y de las transacciones en cada repositorio, y facilitó agregar nuevas entidades sin reescribir la lógica de persistencia.

En cuanto al manejo de transacciones, cada operación abre su propio “EntityManager”, ejecuta la operación dentro de una transacción explícita y hace rollback ante cualquier excepción, cerrando siempre el recurso en un bloque “finally”. Este enfoque prioriza la integridad de los datos por sobre la brevedad del código, algo especialmente importante en un proyecto donde no hay un contenedor (como Spring) que gestione ese ciclo de vida automáticamente.

La baja lógica se implementó como un campo booleano (eliminado) en la clase base, en lugar de eliminar físicamente los registros. Esto permite conservar el historial de pedidos y productos aun cuando el usuario los “elimine” desde el menú, lo cual es coherente con un sistema de ventas donde borrar un producto con pedidos asociados generaría inconsistencias.

2.2 Frontend

La aplicación se organizó por páginas (auth, store, client, admin) y utilidades compartidas (api, auth, cart, orders, navigate), separando la vista de la lógica reutilizable. Los tipos de dominio (product.ts, order.ts, IUser.ts, category.ts, Rol.ts) se definieron en TypeScript para que el compilador detecte inconsistencias entre los datos y su uso en pantalla antes de ejecutar la aplicación. El carrito y la sesión del usuario se persisten en “localStorage”, para permitir mantener el estado entre recargas sin necesidad de un backend de sesión.

2.3 Estructura del proyecto

FoodStore/

```
├── backend/      # App de consola en Java (sin servidor web)
│   ├── src/main/
│   │   ├── java/com/tp/jpa/
│   │   │   ├── model/  # Entidades JPA (tablas del dominio)
│   │   │   │   ├── enums/ # Valores fijos usados por las entidades
│   │   │   │   │   ├── Estado.java      # Estado del pedido (pendiente, entregado, etc.)
│   │   │   │   │   ├── FormaPago.java   # Efectivo, tarjeta, etc.
│   │   │   │   │   └── Rol.java          # ADMIN / USUARIO
│   │   │   │   ├── Base.java            # Clase padre: id, eliminado, createdAt
│   │   │   │   ├── Calculable.java      # Interfaz: obliga a implementar calcularTotal()
│   │   │   │   ├── Categoria.java       # Categoría de productos
│   │   │   │   ├── DetallePedido.java   # Línea de producto dentro de un pedido
│   │   │   │   ├── Pedido.java          # Cabecera de una compra
│   │   │   │   ├── Producto.java        # Producto en venta, con stock y precio
│   │   │   │   └── Usuario.java         # Cliente o admin, con rol
│   │   │   ├── repository/ # Acceso a datos (CRUD por entidad)
│   │   │   │   ├── BaseRepository.java  # CRUD genérico, lo heredan todos
│   │   │   │   ├── CategoriaRepository.java
│   │   │   │   ├── PedidoRepository.java
│   │   │   │   ├── ProductoRepository.java
│   │   │   │   └── UsuarioRepository.java
│   │   │   └── util/
│   │   │       ├── JPAUtil.java        # Crea y cierra el EntityManagerFactory
│   │   │       └── Main.java           # Punto de entrada: menú y flujo de la consola
│   │   └── resources/META-INF/
│   │       └── persistence.xml         # Config de conexión a H2 + Hibernate
│   ├── build.gradle                   # Dependencias (Hibernate, H2, Lombok)
│   ├── gradlew / gradlew.bat          # Ejecutan Gradle sin instalarlo aparte
│   ├── settings.gradle                # Nombre del proyecto Gradle
│   └── README.md                      # Cómo correr el backend
```

```

└─ frontend/    # App web con Vite + TypeScript
  └─ public/data/ # JSON semilla, se cargan la primera vez
  └─ src/
    └─ assets/images/ # Recursos visuales estáticos
    └─ pages/          # Una carpeta por pantalla (html+css+ts)
      └─ admin/
        └─ adminHome/ # Dashboard del panel admin
        └─ categories/ # ABM de categorías
        └─ orders/     # Gestión de pedidos (admin)
        └─ products/   # ABM de productos
      └─ auth/
        └─ login/      # Formulario de inicio de sesión
        └─ register/   # Alta de usuario nuevo
      └─ client/
        └─ orders/     # Historial de pedidos del usuario
      └─ store/
        └─ cart/       # Carrito y checkout
        └─ home/       # Catálogo, búsqueda y filtros
        └─ productDetail/ # Detalle de un producto puntual
      └─ styles/       # CSS global y compartido
      └─ types/        # Contratos TypeScript del dominio
        └─ category.ts
        └─ IUser.ts
        └─ order.ts
        └─ product.ts
        └─ Rol.ts
      └─ utils/        # Lógica reutilizable, sin UI
        └─ adminData.ts # Altas/bajas desde el panel admin
        └─ adminLayout.ts # Header/sidebar común del panel admin
        └─ api.ts       # Único punto de acceso a datos (fetch + JSON)
        └─ auth.ts      # Login, logout, validación de sesión
        └─ cart.ts      # Agregar/quitar del carrito
        └─ localStorage.ts # Wrapper de lectura/escritura en el navegador
        └─ navigate.ts  # Redirecciones entre páginas
        └─ orders.ts    # Crear y consultar pedidos
      └─ main.ts       # Route guard: valida rol antes de cada página
  └─ index.html       # HTML base que carga main.ts
  └─ package.json     # Dependencias y scripts del proyecto (dev, build)
  └─ pnpm-lock.yaml   # Config del bundler
  └─ tsconfig.json    # Config del compilador TS
  └─ vite.config.ts   # Config del bundler
  └─ README.md        # Cómo correr el frontend

```

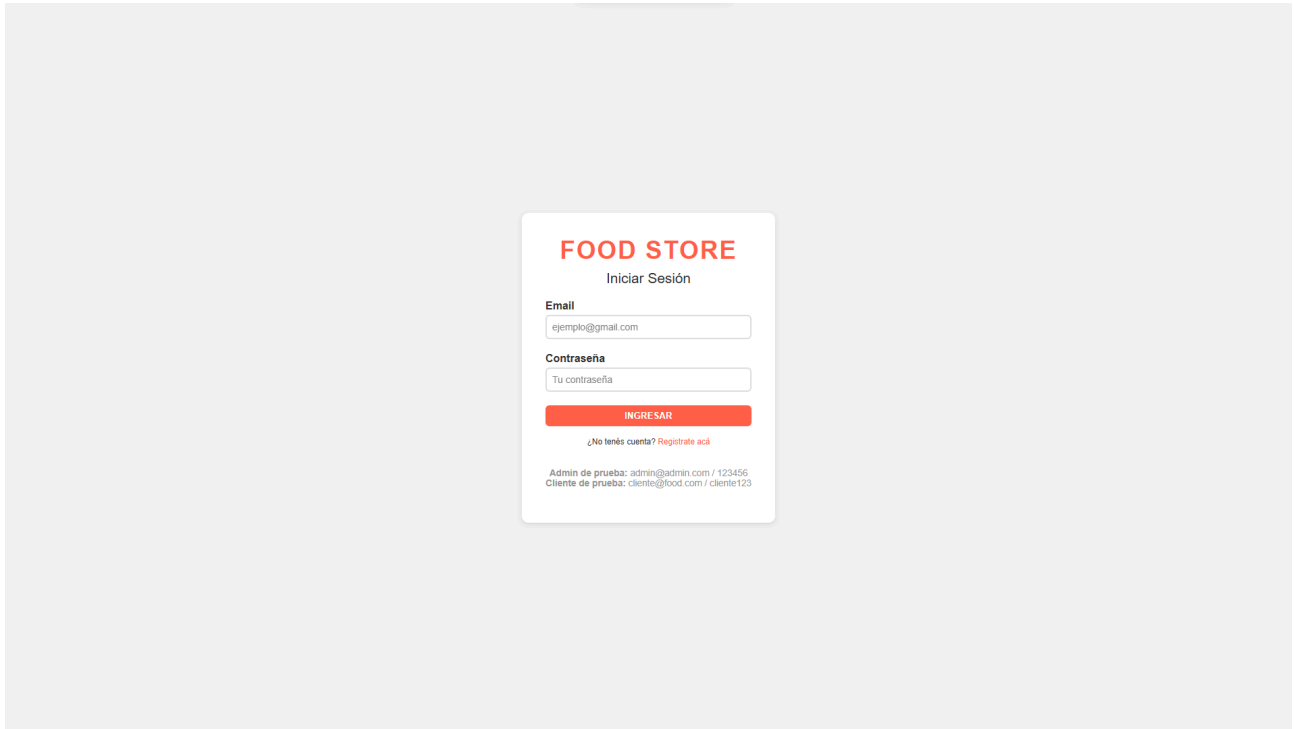
Esta separación permite modificar la interfaz sin tocar la capa de persistencia y viceversa, y facilita agregar entidades o vistas nuevas sin reescribir código existente.

3. Capturas de pantalla

A continuación se presentan capturas de pantalla de las diferentes vistas de la web de Food Store:

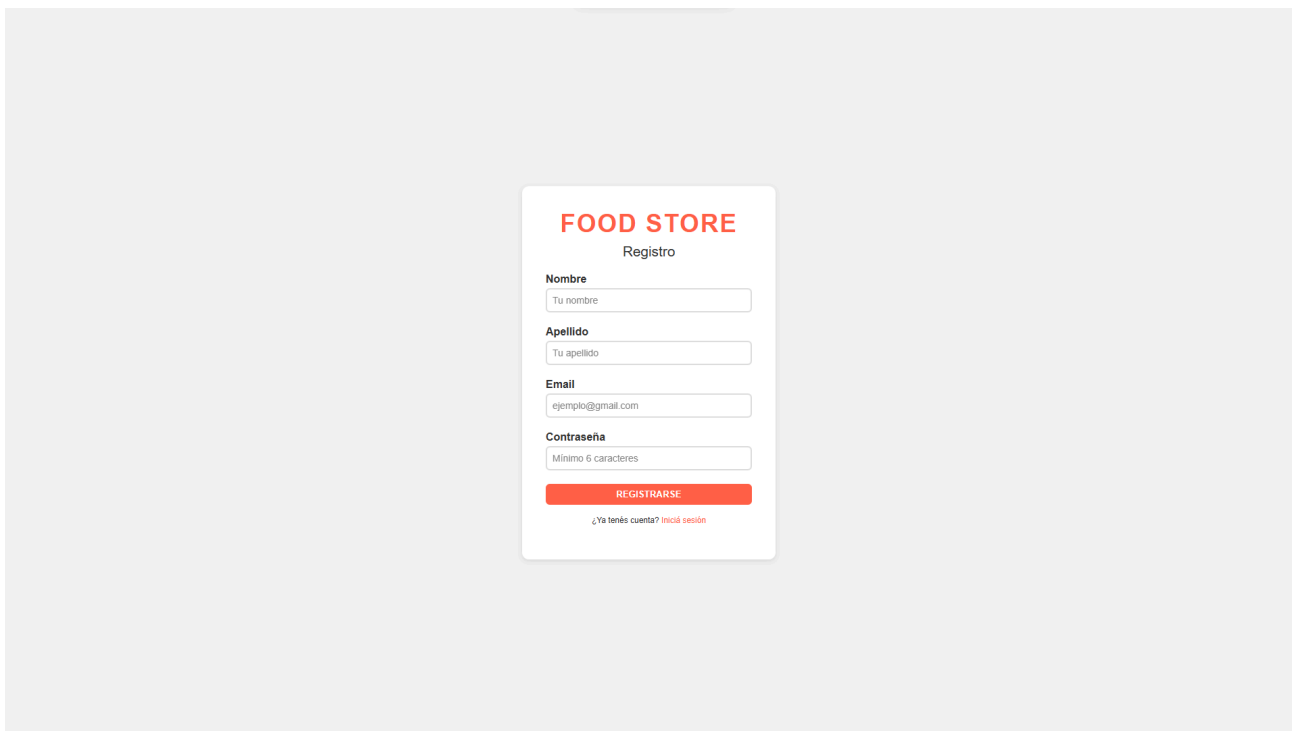
3.1 Frontend

3.1.1 Login:



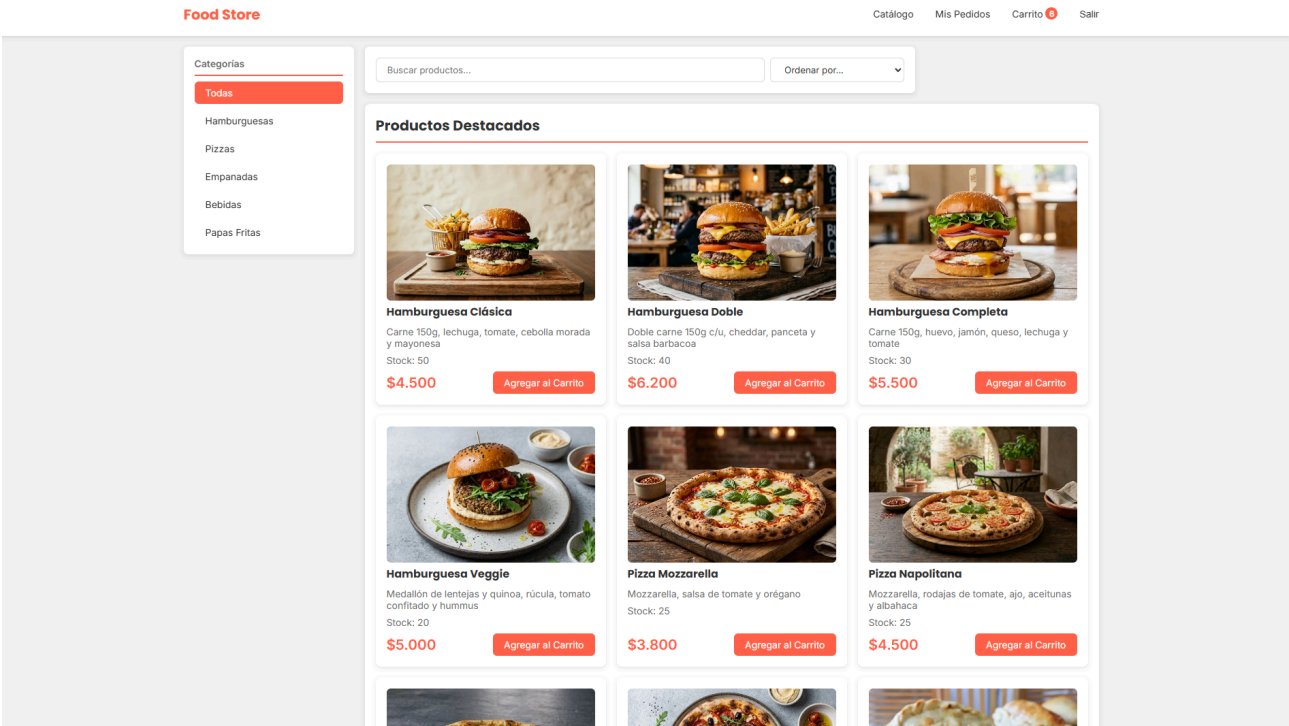
The screenshot shows a login form for 'FOOD STORE'. The form is centered on a light gray background. It has a title 'FOOD STORE' in red, followed by 'Iniciar Sesión' in black. There are two input fields: 'Email' with the placeholder 'ejemplo@gmail.com' and 'Contraseña' with the placeholder 'Tu contraseña'. Below these is a red button labeled 'INGRESAR'. Under the button is a link: '¿No tenés cuenta? [Regístrate acá](#)'. At the bottom, there is small text: 'Admin de prueba: admin@admin.com / 123456' and 'Cliente de prueba: cliente@food.com / cliente123'.

3.1.2 Registro:

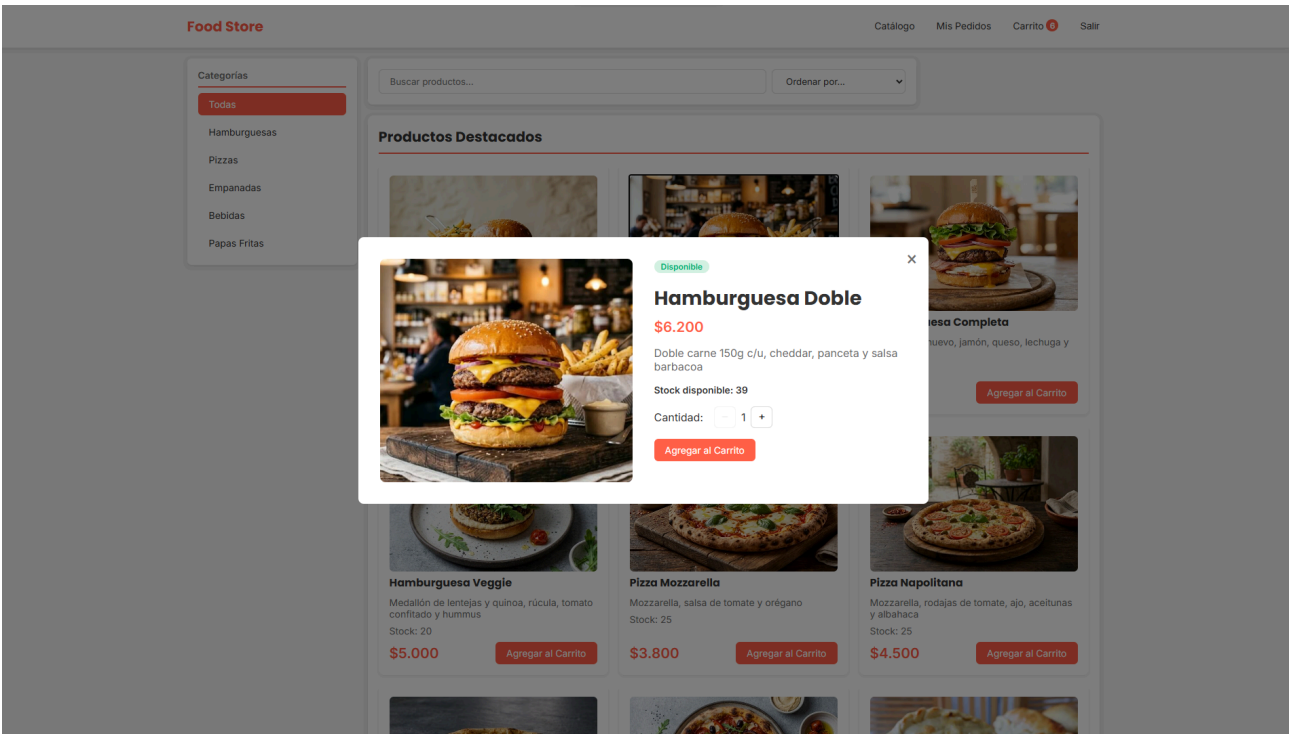


The screenshot shows a registration form for 'FOOD STORE'. The form is centered on a light gray background. It has a title 'FOOD STORE' in red, followed by 'Registro' in black. There are four input fields: 'Nombre' with placeholder 'Tu nombre', 'Apellido' with placeholder 'Tu apellido', 'Email' with placeholder 'ejemplo@gmail.com', and 'Contraseña' with placeholder 'Mínimo 6 caracteres'. Below these is a red button labeled 'REGISTRARSE'. Under the button is a link: '¿Ya tenés cuenta? [Iniciá sesión](#)'.

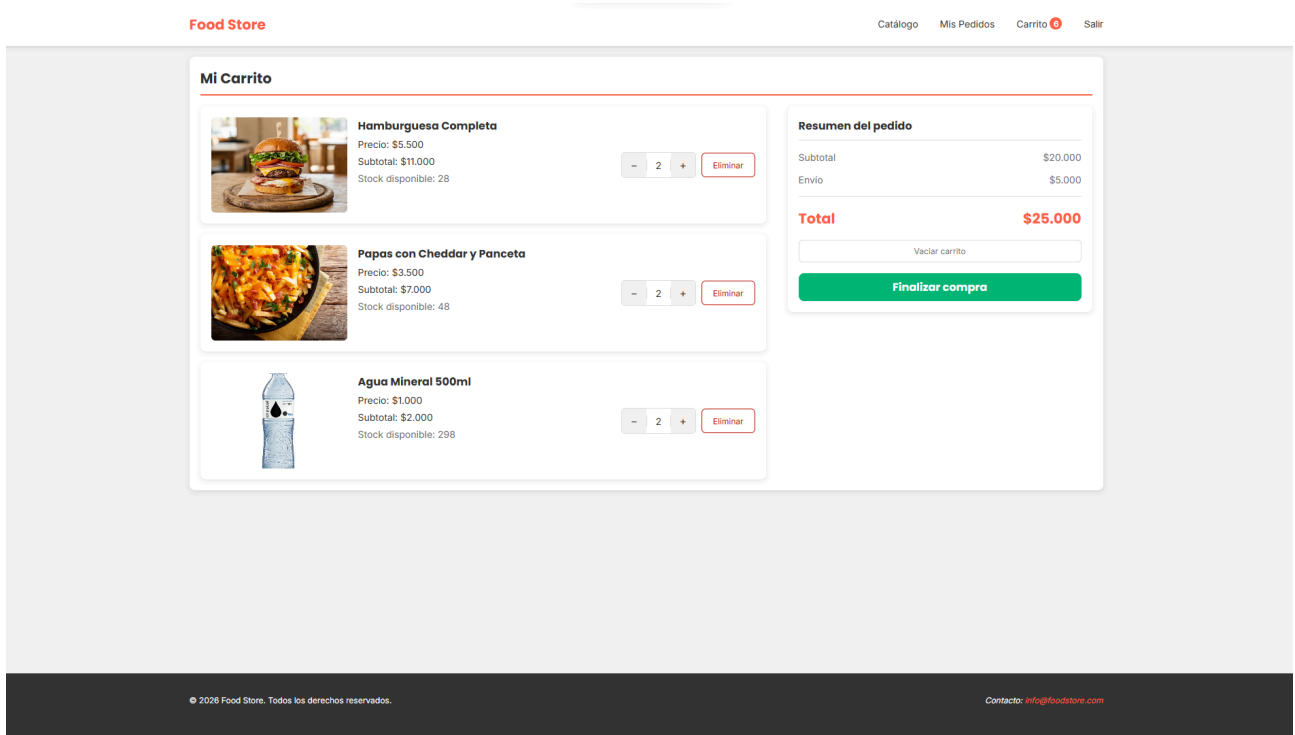
3.1.3 Catálogo:



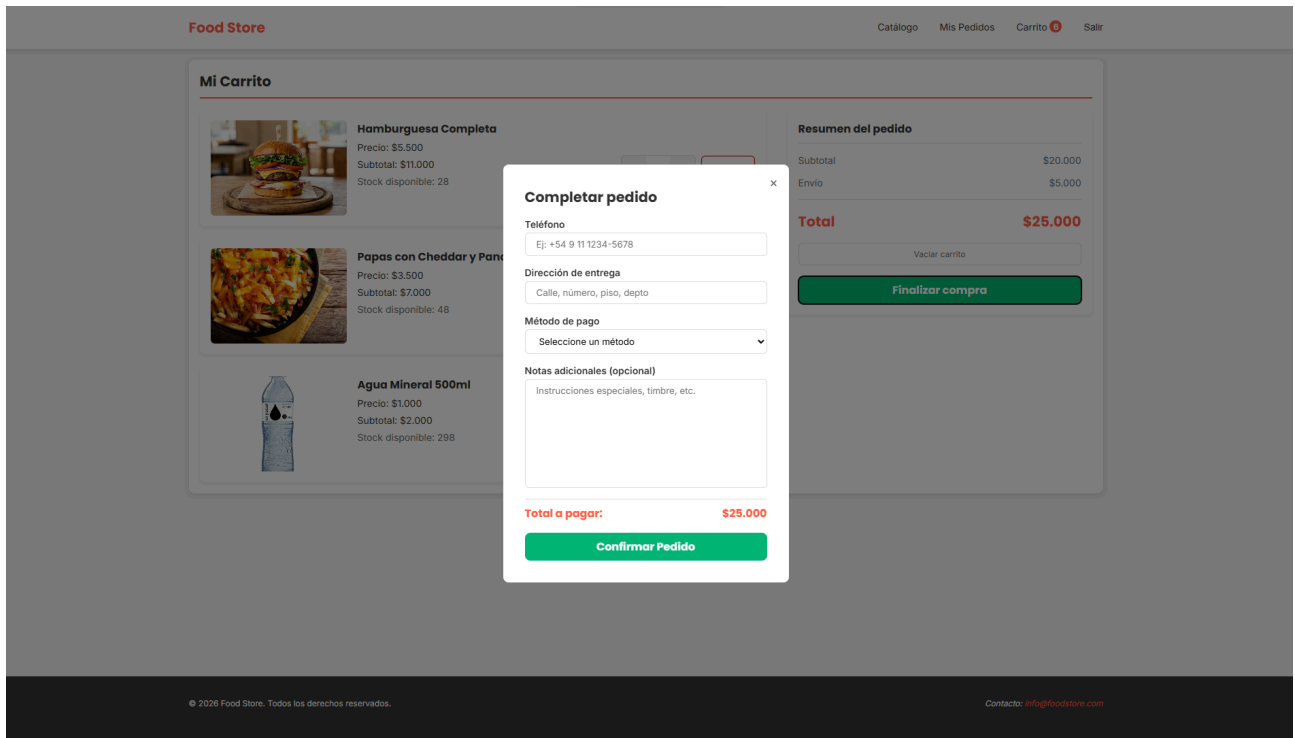
3.1.4 Detalle Producto:



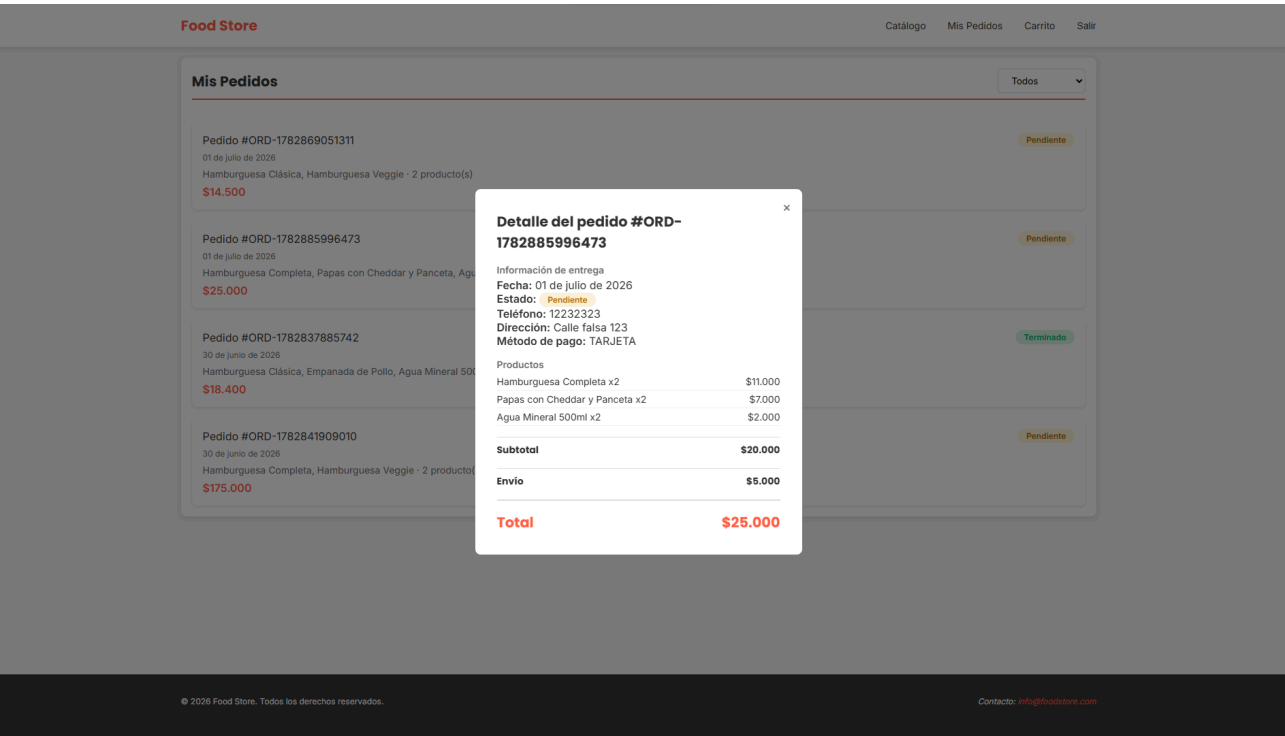
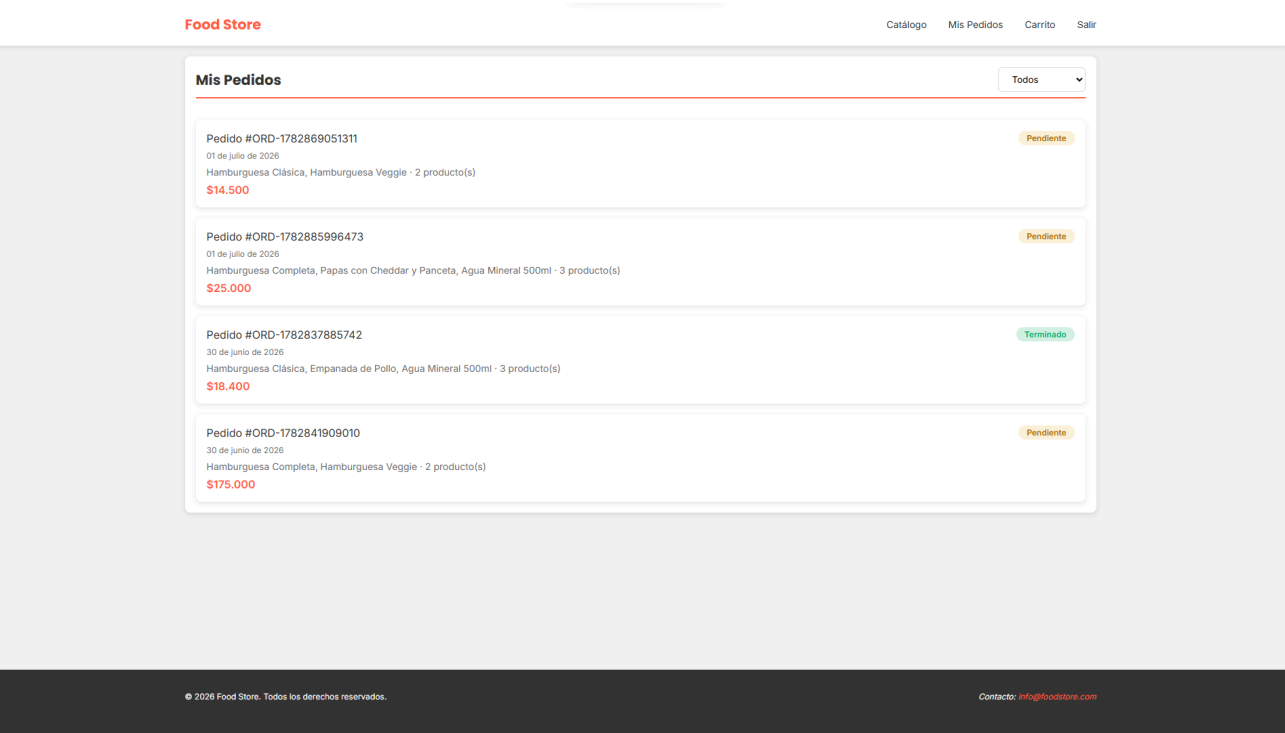
3.1.5 Carrito:



3.1.6 Confirmación del Pedido:



3.1.7 Pedidos del Cliente:



3.1.8 Home Admin:

Food Store Admin

Ver Tienda

Salir

Administración

Dashboard

Categorías

Productos

Pedidos

Panel de Administración

Categorías

5

Productos

19

Pedidos

22

Disponibles

19

Resumen Rápido

Categorías activas	5
Productos activos / inactivos	19 / 7
Pedidos PENDIENTE	10
Pedidos CONFIRMADO	4
Pedidos TERMINADO	2
Pedidos CANCELADO	4

Panel de Administración — Food Store v1.0

3.1.9 Administración Categorías:

Food Store Admin

Ver Tienda

Salir

Administración

Dashboard

Categorías

Productos

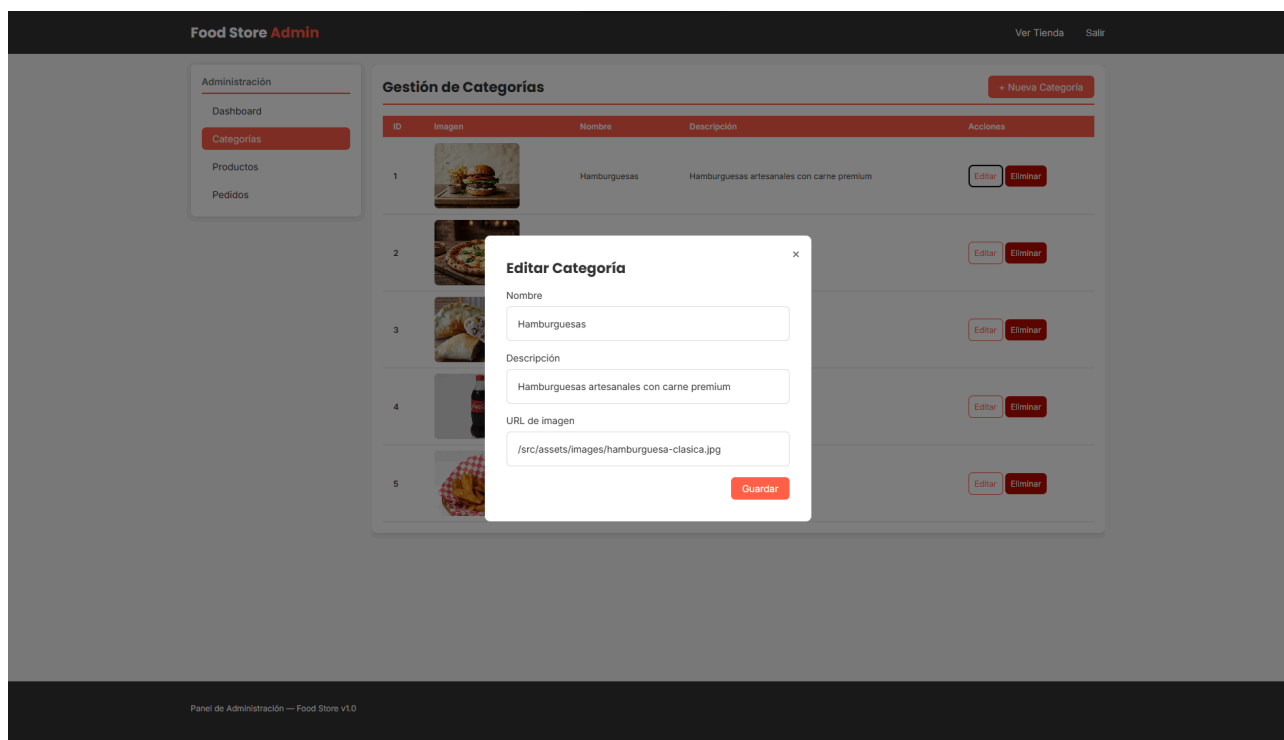
Pedidos

Gestión de Categorías

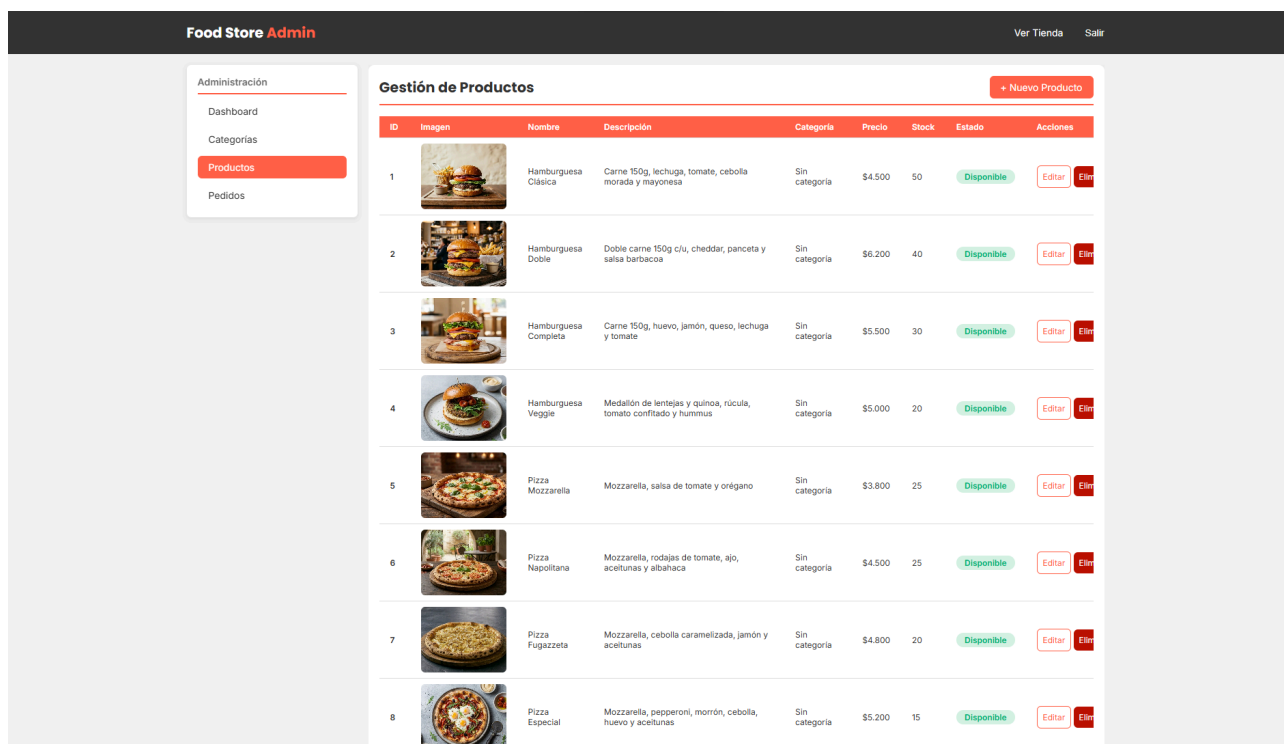
+ Nueva Categoría

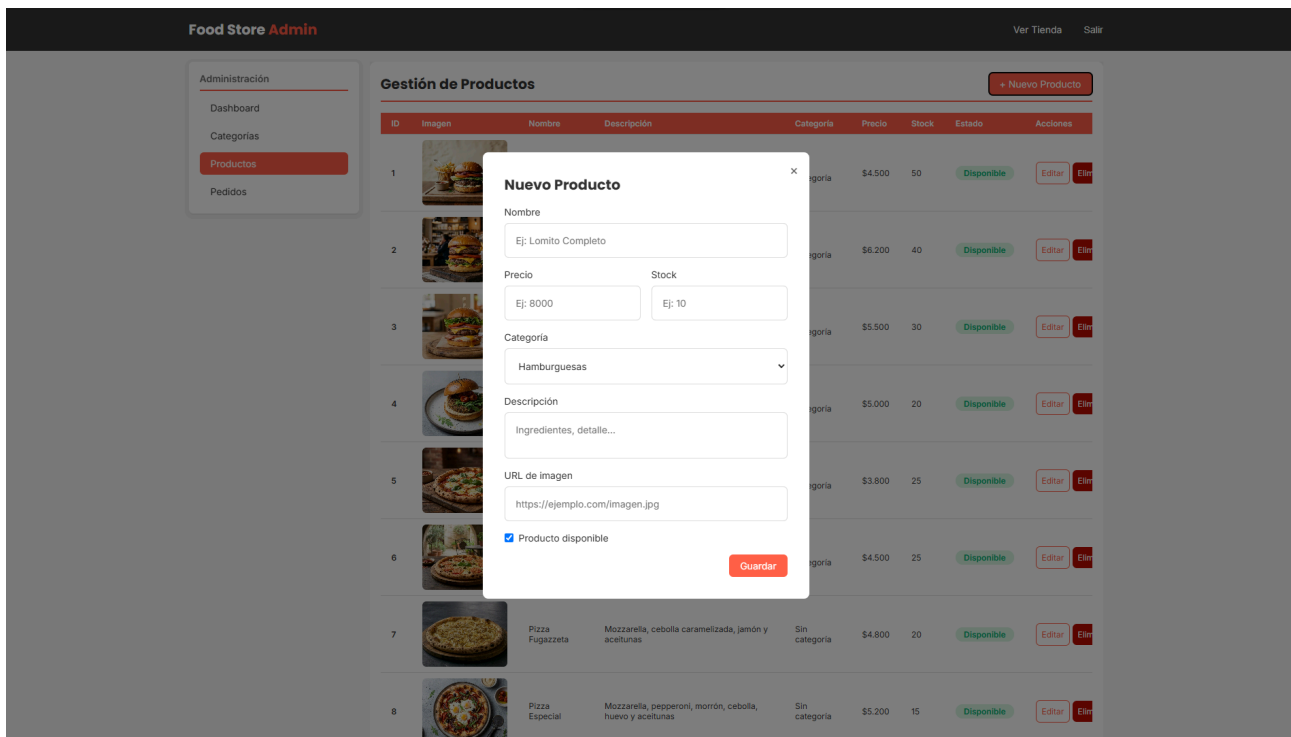
ID	Imagen	Nombre	Descripción	Acciones
1		Hamburguesas	Hamburguesas artesanales con carne premium	Editar Eliminar
2		Pizzas	Pizzas al horno de barro	Editar Eliminar
3		Empanadas	Empanadas caseras horneadas	Editar Eliminar
4		Bebidas	Bebidas frías y calientes	Editar Eliminar
5		Papas Fritas	Acompañamientos crujientes	Editar Eliminar

Panel de Administración — Food Store v1.0

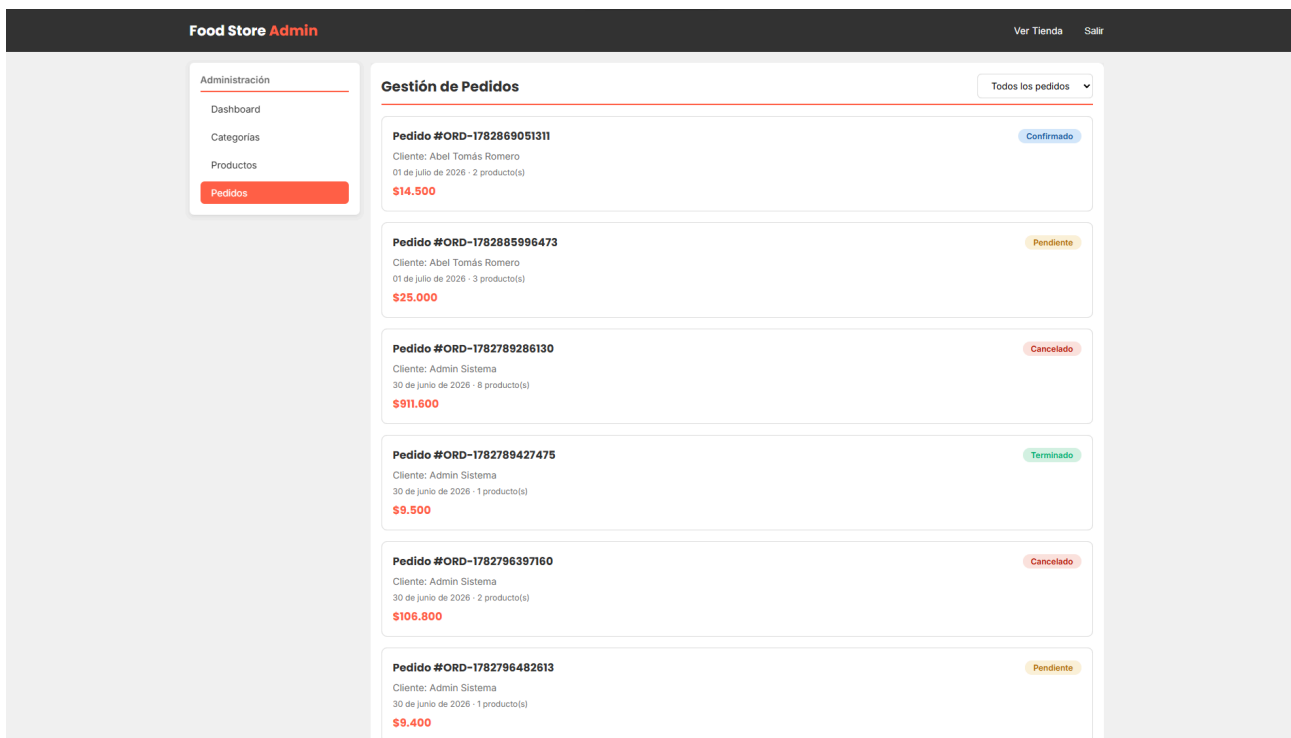


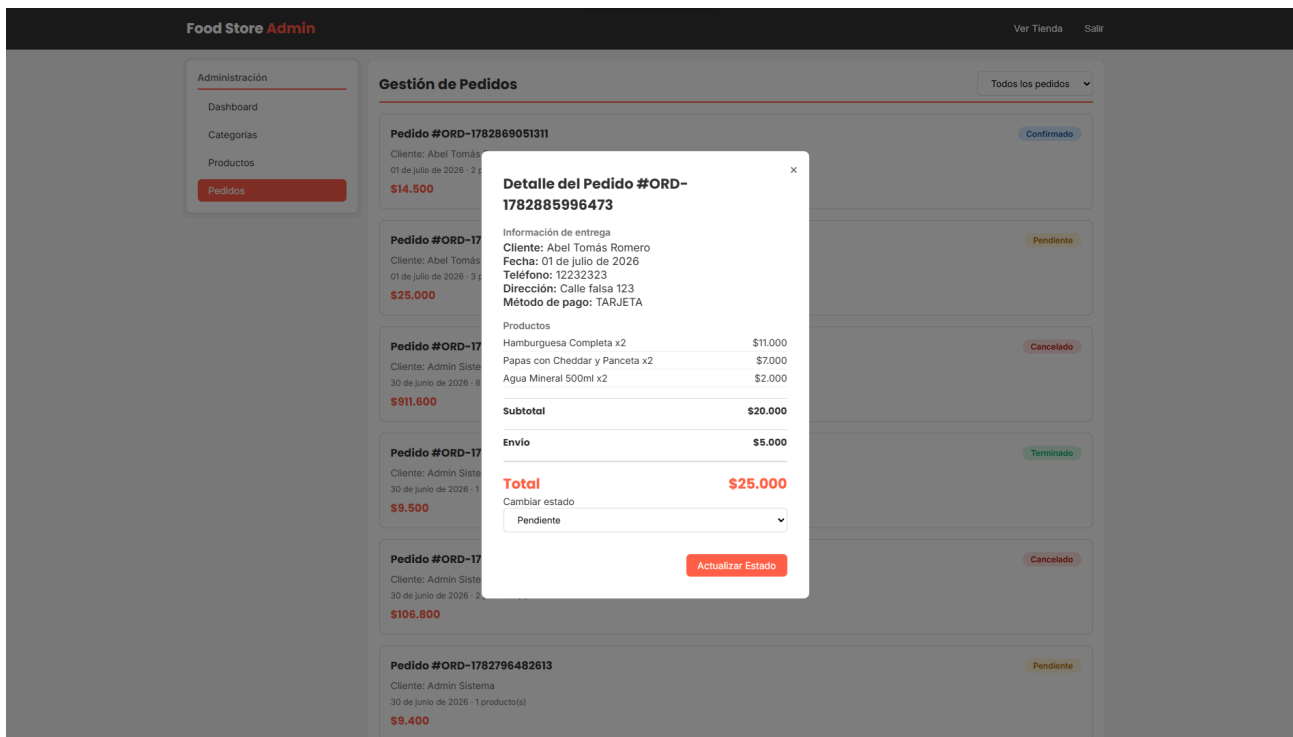
3.1.10 Administración Productos:





3.1.11 Administración Pedidos:





3.2 Backend

3.2.1 Menú principal:

```
=====
FOOD STORE - Menú principal
=====
1. Gestión Categorías
2. Gestión Productos
3. Gestión Usuarios
4. Gestión Pedidos
5. Reportes
0. Salir
-----
Opción:
```

3.2.2 Menú Gestión Categorías:

```
----- Gestión Categorías -----
1. Alta de categoría
2. Modificar categoría
3. Baja lógica de categoría
4. Listar categorías activas
0. Volver al menú principal
-----
Opción:
```

3.2.3 Alta de categoría:

```
---- Alta de categoría ----
Nombre: Postres
Descripción (opcional): Diferentes tipos de comidas dulces

-> [+] Categoría guardada con ID: 1
```

3.2.4 Modificar categoría:

```
---- Modificar categoría ----

---- Categorías activas ----
ID   Nombre                               Descripción
-----
1     Postres                             Diferentes tipos de comidas dulces

Ingrese ID de la categoría a modificar (0 para cancelar): 1
Nombre actual: Postres
Nuevo nombre (Enter para mantener):
Descripción actual: Diferentes tipos de comidas dulces
Nueva descripción (Enter para mantener): Comidas dulces para disfrutar

-> [+] Categoría modificada correctamente.
```

3.2.5 Baja lógica de categoría:

```
---- Baja lógica de categoría ----

---- Categorías activas ----
ID   Nombre                               Descripción
-----
1     Postres                             Comidas dulces para disfrutar
2     Carnes                              Diferentes tipos de carnes y cortes

ID de la categoría a dar de baja (0 para cancelar): 2
```

3.2.6 Listar categorías activas:

```
---- Categorías activas ----
ID   Nombre                               Descripción
-----
1     Postres                             Comidas dulces para disfrutar
2     Carnes                              Diferentes tipos de carnes y cortes
```

3.2.7 Menú Gestión Productos:

```
----- Gestión Productos -----
1. Alta de producto
2. Modificar producto
3. Baja lógica de producto
4. Listar productos activos
0. Volver al menú principal
-----
```

3.2.8 Alta producto:

```
---- Alta producto ----

---- Categorías activas ----
ID   Nombre                               Descripción
-----
1     Postres                             Comidas dulces para disfrutar
3     Bebidas                             Agua, jugos frutales, gaseosas, etc.
4     Hamburguesas                         Hamburguesas de todo tipo

ID de categoría (0 para cancelar): 1
Nombre del producto: Helado 1kg
Precio: 10000
Descripción (opcional): Tres tipos de sabores a elegir
Stock: 10
Imagen (opcional):
Disponible (S/N, default S): S

-> [+] Producto guardado con ID: 1 | Categoría: Postres
```

3.2.9 Modificar producto:

```
---- Modificar producto ----

---- Productos activos ----
ID      Nombre                Descripción                Precio    Stock    Disponible  Categoría
-----
1       Helado 1kg             Tres tipos de sabores a elegir  10000,00  10      Si          Postres
2       Hamburguesa Clásica      Carne 150g, lechuga, tomate y ce... 8000,00  30      Si          Hamburguesas
3       Sprite 1L                  Botella de 1l de Sprite         3000,00  40      Si          Bebidas

ID del producto a modificar (0 para cancelar): 1
Nombre actual: Helado 1kg
Nuevo nombre (Enter para mantener):
Descripción actual: Tres tipos de sabores a elegir
Nueva descripción (Enter para mantener): Tres sabores a elegir en un bote de 1kg
Precio actual: 10000.0
Nuevo precio (Enter para mantener):
Stock actual: 10
Nuevo stock (Enter para mantener): 20
Imagen actual: null
Nueva imagen (Enter para mantener):
Disponible actualmente: Si
¿Disponible? (S/N, Enter para mantener):

-> [+] Producto modificado correctamente.
```

3.2.10 Baja lógica de producto:

```
---- Baja producto ----

---- Productos activos ----
ID      Nombre                Descripción                Precio    Stock    Disponible  Categoría
-----
1       Helado 1kg             Tres sabores a elegir en un bote... 10000,00  20      Si          Postres
2       Hamburguesa Clásica      Carne 150g, lechuga, tomate y ce... 8000,00  30      Si          Hamburguesas
3       Sprite 1L                  Botella de 1l de Sprite         3000,00  40      Si          Bebidas
4       Cerveza Quilmes            Lata pequeña de cerveza Quilmes    2000,00  30      No          Bebidas

ID del producto a dar de baja (0 para cancelar): 4

-> [+] Producto 'Cerveza Quilmes' dado de baja correctamente.
```

3.2.11 Listar productos activos:

```
---- Productos activos ----
ID      Nombre                Descripción                Precio    Stock    Disponible  Categoría
-----
1       Helado 1kg             Tres sabores a elegir en un bote... 10000,00  20      Si          Postres
2       Hamburguesa Clásica      Carne 150g, lechuga, tomate y ce... 8000,00  30      Si          Hamburguesas
3       Sprite 1L                  Botella de 1l de Sprite         3000,00  40      Si          Bebidas
```

3.2.12 Menú Gestión Usuarios:

```
----- Gestión Usuarios -----
1. Alta de usuario
2. Modificar usuario
3. Baja lógica de usuario
4. Listar usuarios activos
5. Buscar por mail
0. Volver al menú principal
-----
Opción: 1
```

3.2.13 Alta de usuario:

```
---- Alta de usuario ----
Nombre: Abel Tomás
Apellido: Romero
Mail: abeltomasr98@gmail.com
Celular (opcional): 123456789
Contraseña: admin98
Rol: 1. ADMIN  2. USUARIO
Opción: 1

-> [+] Usuario guardado con ID: 1
```

3.2.14 Modificar usuario:

```
---- Modificar usuario ----

---- Usuarios activos ----
ID      Nombre completo      Mail                      Rol
-----
1       Abel Tomás Romero      abeltomasr98@gmail.com    ADMIN
2       Celeste Torcivia        pequis@mail.com           USUARIO

ID del usuario a modificar (0 para cancelar): 2
Nombre actual: Celeste
Nuevo nombre (Enter para mantener):
Apellido actual: Torcivia
Nuevo apellido (Enter para mantener): Torcivia de Romero
Mail actual: pequis@mail.com
Nuevo mail (Enter para mantener):
Celular actual: 987654321
Nuevo celular (Enter para mantener):
Contraseña actual: [oculta]
Nueva contraseña (Enter para mantener):

-> [+] Usuario modificado correctamente.
```

3.2.15 Baja lógica de usuario:

```
---- Baja lógica de usuario ----

---- Usuarios activos ----
ID      Nombre completo      Mail                      Rol
-----
1       Abel Tomás Romero      abeltomasr98@gmail.com    ADMIN
2       Celeste Torcivia de Romero  pequis@mail.com           USUARIO
3       Fulano Rodríguez          ejemplo@mail.com           USUARIO

ID del usuario a dar de baja (0 para cancelar): 3

-> [+] Usuario 'Fulano Rodríguez' dado de baja correctamente.
Sus pedidos permanecen en el sistema.
```

3.2.16 Listar usuarios activos:

```
---- Usuarios activos ----
ID      Nombre completo      Mail                      Rol
-----
1       Abel Tomás Romero      abeltomasr98@gmail.com    ADMIN
2       Celeste Torcivia de Romero  pequis@mail.com           USUARIO
```


3.2.17 Buscar usuario por mail:

```
---- Buscar usuario por mail ----
Mail: abeltomasr98@gmail.com

---- Datos del usuario -----
- ID: 1
- Nombre: Abel Tomás Romero
- Mail: abeltomasr98@gmail.com
- Celular: 123456789
- Rol: ADMIN
```

3.2.18 Menú Gestión Pedidos:

```
----- Gestión Pedidos -----
1. Alta de pedido
2. Cambiar estado de pedido
3. Baja lógica de pedido
4. Listar pedidos activos
5. Pedidos por usuario
6. Pedidos por estado
0. Volver al menú principal
-----
Opción: |
```

3.2.19 Alta de pedido:

```
---- Alta de pedido ----

---- Usuarios activos ----
ID   Nombre completo      Mail                      Rol
-----
1    Abel Tomás Romero    abeltomasr98@gmail.com   ADMIN
2    Celeste Torcivia de Romero    pequis@mail.com          USUARIO

ID del usuario (0 para cancelar): 2
Forma de pago: 1. TARJETA  2. TRANSFERENCIA  3. EFECTIVO
Opción: 1
```

```
---- Productos disponibles ----
ID   Nombre                Precio   Stock
-----
1    Helado 1kg             10000,00  20
2    Hamburguesa Clásica    8000,00   30
3    Sprite 1L              3000,00   40
```

```
ID del producto a agregar (0 para terminar): 3
Cantidad (stock disponible: 40): 1
-> Agregado: Sprite 1L x1
¿Agregar otro producto? (S/N): n
```

```
==== Pedido creado =====
- ID: 1
- Fecha: 2026-07-02
- Usuario: Celeste Torcivia de Romero
- Forma pago: TARJETA
- Estado: PENDIENTE

---- Detalle: ----
Producto      Cant.   P. Unit.   Subtotal
- Helado 1kg  2       $10000,00  $20000,00
- Sprite 1L   1       $3000,00   $3000,00
- Hamburguesa Clásica  2       $8000,00   $16000,00

Total: $39000,00
=====
```

3.2.20 Cambiar estado de pedido:

```
---- Cambiar estado de pedido ----

---- Pedidos activos ----
ID   Fecha       Estado      FormaPago    Usuario                               Total
-----
1    2026-07-02    PENDIENTE   TARJETA      Celeste Torcivia de Romero          $39000,00
2    2026-07-02    PENDIENTE   EFECTIVO     Abel Tomás Romero                   $16000,00

ID del pedido (0 para cancelar): 1
Estado actual: PENDIENTE
Estado: 1. PENDIENTE  2. CONFIRMADO  3. TERMINADO  4. CANCELADO
Opción: 2

-> Pedido con ID 1 actualizado a: CONFIRMADO
```

3.2.21 Baja lógica de pedido:

```
---- Baja lógica de pedido ----

---- Pedidos activos ----
ID   Fecha       Estado      FormaPago    Usuario                               Total
-----
1    2026-07-02    CONFIRMADO  TARJETA      Celeste Torcivia de Romero          $39000,00
2    2026-07-02    PENDIENTE   EFECTIVO     Abel Tomás Romero                   $16000,00

ID del pedido a dar de baja (0 para cancelar): 2

-> Pedido con ID 2 dado de baja. Total: $16000,00
El stock de los productos NO fue restaurado.
```

3.2.22 Listar pedidos activos:

```
---- Pedidos activos ----
ID   Fecha       Estado      FormaPago    Usuario                               Total
-----
1    2026-07-02    CONFIRMADO  TARJETA      Celeste Torcivia de Romero          $39000,00
3    2026-07-02    PENDIENTE   TRANSFERENCIA Abel Tomás Romero                   $36000,00
```

3.2.23 Pedidos por usuario:

```
---- Pedidos por usuario ----

---- Usuarios activos ----
ID   Nombre completo      Mail                               Rol
-----
1    Abel Tomás Romero     abeltomasr98@gmail.com          ADMIN
2    Celeste Torcivia de Romero pequis@mail.com                 USUARIO

ID del usuario (0 para cancelar): 2

---- Pedidos de 'Celeste Torcivia de Romero':
ID   Fecha       Estado      FormaPago    Total
-----
1    2026-07-02    CONFIRMADO  TARJETA      $39000,00
```

3.2.24 Pedidos por estado:

```
---- Pedidos por estado ----
Estado: 1. PENDIENTE  2. CONFIRMADO  3. TERMINADO  4. CANCELADO
Opción: 1

---- Pedidos con estado 'PENDIENTE':
ID   Fecha       Usuario                               Total
-----
3    2026-07-02    Abel Tomás Romero                   $36000,00
```

3.2.25 Menú Reportes:

```
----- Reportes -----
1. Productos por categoría
2. Pedidos por usuario
3. Pedidos por estado
4. Total facturado
0. Volver al menú principal
-----
Opción: |
```

3.2.26 Reporte de productos por categoría:

```
---- Reporte: Productos por categoría ----

---- Categorías activas ----
ID   Nombre                Descripción
-----
1    Postres                Comidas dulces para disfrutar
3    Bebidas                 Agua, jugos frutales, gaseosas, etc.
4    Hamburguesas            Hamburguesas de todo tipo

ID de categoría (0 para cancelar): 1

Productos activos en 'Postres':
ID   Nombre                Precio   Stock
-----
1    Helado 1kg              10000,00  16
```

3.2.27 Reporte de pedidos por usuario:

```
---- Reporte: Pedidos por usuario ----

---- Usuarios activos ----
ID   Nombre completo        Mail                Rol
-----
1    Abel Tomás Romero      abeltomasr98@gmail.com  ADMIN
2    Celeste Torcivia de Romero  pequis@mail.com        USUARIO

ID del usuario (0 para cancelar): 1

Pedidos de Abel Tomás Romero:
ID   Fecha      Estado    FormaPago    Total
-----
3    2026-07-02  PENDIENTE TRANSFERENCIA $36000,00
```

3.2.28 Reporte de pedidos por estado:

```
---- Reporte: Pedidos por estado ----
Estado: 1. PENDIENTE  2. CONFIRMADO  3. TERMINADO  4. CANCELADO
Opción: 2

Pedidos con estado CONFIRMADO:
ID   Fecha      Usuario                Total
-----
1    2026-07-02  Celeste Torcivia de Romero  $39000,00
```

3.2.29 Reporte del total facturado:

```
---- Reporte: Total facturado ----
Total facturado (pedidos TERMINADOS): $39000.00
```

4. Dificultades y soluciones

En el **backend**, el manejo del “EntityManager” y la sincronización de los datos contra H2 me generaron inconsistencias al probar operaciones de alta y actualización sobre entidades relacionadas (por ejemplo, pedidos con sus detalles). El problema se resolvió ajustando el ciclo de vida de las transacciones, abriendo y cerrando el “EntityManager” por operación, y revisando el uso correcto de “persist” frente a “merge” según si la entidad ya tenía id asignado.

En el **frontend**, mi principal inconveniente fue lograr que la estructura de los archivos JSON de datos de prueba coincidiera con los tipos TypeScript definidos para cada entidad, lo que producía errores de tipado al renderizar las vistas. Se resolvió centralizando la carga y transformación de los datos en una capa de utilidades (api.ts y utilidades relacionadas), que actúa como punto único de acceso a los datos y garantiza que lo que llega a las páginas ya esté validado y tipado correctamente.

5. Herramientas utilizadas

- **Java 17:** lenguaje backend.
- **Gradle:** build y gestión de dependencias.
- **JPA (Jakarta Persistence) + Hibernate 6:** mapeo objeto-relacional y persistencia.
- **H2 Database:** base de datos embebida en archivo.
- **Lombok:** reducción de código repetitivo en entidades.
- **Vite:** bundler y servidor de desarrollo del frontend.
- **TypeScript:** tipado del dominio en el cliente.
- **HTML / CSS:** interfaz de usuario.
- **IntelliJ IDEA / VSCode:** editores de código
- **Claude:** corrección de errores y ayuda a redacción de los informes y README.

6. Bibliografía / Webgrafía

- **Java – Documentación oficial:** <https://docs.oracle.com/en/java/javase/17/>
- **Jakarta Persistence (JPA) – Especificación:** <https://jakarta.ee/specifications/persistence/>
- **Hibernate ORM – Documentación:** <https://hibernate.org/orm/documentation/>
- **H2 Database Engine – Documentación:** <https://www.h2database.com/html/main.html>
- **Gradle – Documentación:** <https://docs.gradle.org/>
- **Project Lombok:** <https://projectlombok.org/>
- **Vite – Guía oficial:** <https://vitejs.dev/>
- **TypeScript – Documentación:** <https://www.typescriptlang.org/docs/>